Advanced information, computation, communication I
EPFL - Fall semester 2026-2027

Week 6 — solutions

October 16, 2026

Exercise 1. *Solve the following recurrence relation using generating functions:* $a_n = 4a_{n-1} - 4a_{n-2}$, $n \geq 2$, *with* $a_0 = 1$ *and* $a_1 = 4$.

Multiplying by x^n and summing over all $n \geq 2$ we get

$$\sum_{n\geq 2} a_n x^n = 4\sum_{n\geq 2} a_{n-1}x^n - 4\sum_{n\geq 2} a_{n-2}x^n.$$

Rearranging, reindexing, and letting $A(x) = \sum_{n\geq 0} a_n x^n$ we get

$$A(x) - a_0 - a_1 x = 4x(A(x) - a_0) - 4x^2 A(x).$$

This has the solution

$$A(x) = \frac{a_0 + a_1 x - 4xa_0}{1 - 4x + 4x^2} = \frac{1}{(1-2x)^2}.$$

This already has the form of the partial fraction and note that we have a double root. From the lecture we know that this has the expansion

$$A(x) = \frac{1}{(1-2x)^2} = \sum_{n\geq 0}(n+1)2^n x^n.$$

It is always a good idea to make a sanity check and to plug this back into the recursion. We get

$$(n+1)2^n = 4n2^{n-1} - 4(n-1)2^{n-2}.$$

Dividing by 2^n we get

$$(n+1) = 2n - (n-1) = (n+1)$$

Exercise 2. *Show that* $\sum_{j=1}^{n}(a_j - a_{j-1}) = a_n - a_0$, *where* $a_0, a_1, ..., a_n$ *is a sequence of real numbers. This type of sum is called* ***telescoping****.*

We start by decomposing the difference:

$$\sum_{j=1}^{n}(a_j - a_{j-1}) = \sum_{j=1}^{n} a_j - \sum_{j=1}^{n} a_{j-1}$$

We know that: $\sum_{j=1}^{n} a_j = a_1 + a_2 + ... + a_{n-1} + a_n$ (i)
and that $\sum_{j=1}^{n} a_{j-1} = a_0 + a_1 + ... + a_{n-2} + a_{n-1}$ (ii).
Taking the difference $i - ii$:

$$\sum_{j=1}^{n} a_j - \sum_{j=1}^{n} a_{j-1} = (a_1 + a_2 + ... + a_{n-1} + a_n) - (a_0 + a_1 + ... + a_{n-2} + a_{n-1})$$

$$= \mathbf{a_1} + a_2 + ... + a_{n-1} + a_n - a_0 - \mathbf{a_1} - ... - a_{n-2} - a_{n-1}$$

$$= a_n - a_0$$

Exercise 3. *Sum both sides of the identity $k^2 - (k-1)^2 = 2k - 1$ from $k = 1$ to $k = n$ and use Exercise 2 to find a formula for (1) and (2).*

Let's start with $\sum_{\mathbf{k=1}}^{\mathbf{n}}(\mathbf{2k-1})$.

We know that:

$$k^2 - (k-1)^2 = k^2 - (k^2 - 2k + 1) = 2k - 1 \tag{1}$$

and

$$\sum_{k=1}^{n} a_k - a_{k-1} = a_n - a_0 \tag{2}$$

Using (1), we start with:

$$\sum_{k=1}^{n}(2k-1) = \sum_{k=1}^{n}(k^2 - (k-1)^2)$$

Now, replacing a_k by k^2 and a_{k-1} by $(k-1)^2$, we get:

$$\sum_{k=1}^{n}(k^2 - (k-1)^2) = \sum_{k=1}^{n}(2k-1) = (n^2) - (0)^2 = n^2$$

Let's move on to $\sum_{\mathbf{k=1}}^{\mathbf{n}}(\mathbf{k})$.

We start with $k^2 - (k-1)^2 = 2k - 1$

$$\Rightarrow 2k - 1 = \{k^2 - (k-1)^2\}$$

$$\Rightarrow \sum_{k=1}^{n}(2k-1) = \sum_{k=1}^{n}\{k^2 - (k-1)^2\}$$

$$2\sum_{k=1}^{n} k - \sum_{k=1}^{n} 1 = \sum_{k=1}^{n}\{k^2 - (k-1)^2\}$$

$$2\sum_{k=1}^{n} k - n = \sum_{k=1}^{n}\{k^2 - (k-1)^2\}$$

Now, using Exercise 2: $\sum_{k=1}^{n}(a_k - a_{k-1}) = a_n - a_0$

$$\Rightarrow 2\sum_{k=1}^{n} k - n = n^2 - 0^2 = n^2$$

$$\Rightarrow 2\sum_{k=1}^{n} k = n^2 + n = n(n+1)$$

$$\Rightarrow \sum_{k=1}^{n} k = (\frac{n(n+1)}{2})$$

Exercise 4. *Use Bubble Sort, Selection Sort, and Insertion Sort to sort the following sequence:*

`9, 12,-43, 20, -2, 3, 7, 28, 19.`

1. **Bubble Sort**:

   We have a sequence of length 9. The procedure passes through the entire sequence and compares each pair of consecutive numbers (x_i, x_{i+1}) that are swapped if $x_i > x_{i+1}$ for $1 \leq i < 9$. Hence, the resulting sequences in the end of each passing through are the following:

   ```
   9, -43, 12, -2, 3, 7, 20, 19, 28.
   -43, 9, -2, 3, 7, 12, 19, 20, 28.
   -43, -2, 3, 7, 9, 12, 19, 20, 28.
   -43, -2, 3, 7, 9, 12, 19, 20, 28.
   -43, -2, 3, 7, 9, 12, 19, 20, 28.
   -43, -2, 3, 7, 9, 12, 19, 20, 28.
   -43, -2, 3, 7, 9, 12, 19, 20, 28.
   -43, -2, 3, 7, 9, 12, 19, 20, 28.
   ```

   It is possible to modify bubble sort to keep track of the number of swaps it performs, and in this version of the algorithm we can skip the last four passes.

2. **Selection Sort**:

   We have a sequence of length 9. We start with $i = 1$, i.e. $min = 2$. Then the minimum of $a_2, \ldots, a_9$ is $a_3 = -43 =: a_{min}$. Since $a_1 > a_{min}$ we swap these two and get

   ```
   -43, 12, 9, 20,-2, 3, 7, 28, 19.
   ```

   Now we go to $i = 2$, i.e. $min = 3$. Then the minimum of $a_3, \ldots, a_9$ is $a_5 = -2 =: a_{min}$. Since $a_2 > a_{min}$ we swap these two and get

   ```
   -43, -2, 9, 20, 12, 3, 7, 28, 19.
   ```

   Now we go to $i = 3$, i.e. $min = 4$. Then the minimum of $a_4, \ldots, a_9$ is $a_6 = 3 =: a_{min}$. Since $a_3 > a_{min}$ we swap these two and get

   ```
   -43, -2, 3, 20, 12, 9, 7, 28, 19.
   ```

   Now we go to $i = 4$, i.e. $min = 5$. Then the minimum of $a_5, \ldots, a_9$ is $a_7 = 7 =: a_{min}$. Since $a_4 > a_{min}$ we swap these two and get

   ```
   -43, -2, 3, 7, 12, 9, 20, 28, 19.
   ```

   Now we go to $i = 5$, i.e. $min = 6$. Then the minimum of $a_6, \ldots, a_9$ is $a_6 = 9 =: a_{min}$. Since $a_5 > a_{min}$ we swap these two and get

   ```
   -43, -2, 3, 7, 9, 12, 20, 28, 19.
   ```

   Now we go to $i = 6$, i.e. $min = 7$. Then the minimum of $a_7, \ldots, a_9$ is $a_9 = 19 =: a_{min}$. Since $a_6 < a_{min}$ we do not swap and stay with

   ```
   -43, -2, 3, 7, 9, 12, 20, 28, 19.
   ```

   Now we go to $i = 7$, i.e. $min = 8$. Then the minimum of $a_8, \ldots, a_9$ is $a_9 = 19 =: a_{min}$. Since $a_7 > a_{min}$ we swap these two and get

   ```
   -43, -2, 3, 7, 9, 12, 19, 28, 20.
   ```

   Now we go to $i = 8$, i.e. $min = 9$. Then the minimum of a_9 is $a_9 = 20 =: a_{min}$. Since $a_8 > a_{min}$ we swap these two and get

   ```
   -43, -2, 3, 7, 9, 12, 19, 20, 28.
   ```

3. **Insertion Sort**:

We have a sequence of length 9. We start with $j = 2, i = 1$. The lowest i for which $a_2 \leq a_i$ is $i = 2$ (i.e. this is where the while-loop stops). We set $m = a_2 = 12$. The for-loop is empty and we set $a_i = a_2 := m = 12$. Hence the sequence remains

`9, 12, -43, 20, -2, 3, 7, 28, 19.`

Now we go to $j = 3$. The lowest i for which $a_3 \leq a_i$ is $i = 1$ (i.e. this is where the while-loop stops). We set $m = a_3 = -43$. The for-loop assigns $a_3 := a_2$ and $a_2 := a_1$. Then we set $a_i = a_1 := m = -43$. Hence the new sequence is

`-43, 9, 12, 20, -2, 3, 7, 28, 19.`

Now we go to $j = 4$. The lowest i for which $a_4 \leq a_i$ is $i = 4$ (i.e. this is where the while-loop stops). We set $m = a_4 = 20$. The for-loop is empty. Then we set $a_i = a_4 := m = 20$. Hence the sequence remains

`-43, 9, 12, 20, -2, 3, 7, 28, 19.`

Now we go to $j = 5$. The lowest i for which $a_5 \leq a_i$ is $i = 2$ (i.e. this is where the while-loop stops). We set $m = a_5 = -2$. The for-loop assigns $a_5 := a_4, a_4 := a_3$ and $a_3 := a_2$. Then we set $a_i = a_2 := m = -2$. Hence the new sequence is

`-43, -2, 9, 12, 20, 3, 7, 28, 19.`

Now we go to $j = 6$. The lowest i for which $a_6 \leq a_i$ is $i = 3$ (i.e. this is where the while-loop stops). We set $m = a_6 = 3$. The for-loop assigns $a_6 := a_5, a_5 := a_4, a_4 := a_3$. Then we set $a_i = a_3 := m = 3$. Hence the new sequence is

`-43, -2, 3, 9, 12, 20, 7, 28, 19.`

Now we go to $j = 7$. The lowest i for which $a_7 \leq a_i$ is $i = 4$ (i.e. this is where the while-loop stops). We set $m = a_7 = 7$. The for-loop assigns $a_7 := a_6, a_6 := a_5, a_5 := a_4$. Then we set $a_i = a_4 := m = 7$. Hence the new sequence is

`-43, -2, 3, 7, 9, 12, 20, 28, 19.`

Now we go to $j = 8$. The lowest i for which $a_8 \leq a_i$ is $i = 8$ (i.e. this is where the while-loop stops). We set $m = a_8 = 28$. The for-loop is empty. Then we set $a_i = a_8 := m = 28$. Hence the sequence remains

`-43, -2, 3, 7, 9, 12, 20, 28, 19.`

Now we go to $j = 9$. The lowest i for which $a_9 \leq a_i$ is $i = 7$ (i.e. this is where the while-loop stops). We set $m = a_9 = 19$. The for-loop assigns $a_9 := a_8, a_8 := a_7$. Then we set $a_i = a_7 := m = 19$. Hence the final sequence is

`-43, -2, 3, 7, 9, 12, 19, 20, 28.`

Exercise 5. *Adapt the bubble sort algorithm so that it stops when no swaps are required (when the list is sorted). Express this more efficient version of the algorithm in pseudocode.*

We call the algorithm " improved-bubblesort" and the input is a list of real numbers $a_1, a2, ..., a_n$.

The variable "interchange" will keep track if any interchanges were made during each pass/step. i is the position of an element in the list (we will start from 2 as the first element does not have a preceding term).

$$interchange := 0$$

$$i := 1$$

For every element in the list, we will determine if the elements are in increasing order. If they are not, then we interchange the two elements. If an interchange is made, the the variable "interchange" should be put to 0 (and thus we also need to change the value from 0 in each iteration step).

Algorithm 1 Improved Bubble Sort

```
i := 1
interchange := 0
while i < n and interchange = 0 do
  interchange := 1
  for j = 1 to n − i do
    if a_j > a_{j+1} then
      swap a_j and a_{j+1}
      interchange := 0
    end if
  end for
  i := i + 1
end while
```

Exercise 6. *Two strings are anagrams if each can be formed from the other string by rearranging its characters. Devise an algorithm to determine whether two strings are anagrams*

As the characters of both words should be exactly similar to each other to be anagrams, we can proceed as follows.

First we check if the length of the two strings are equal (as two strings with different length cannot be anagrams).

If they have the same length, then we sort both strings using any sorting algorithm like bubble-sort (the "value" of each character can be defined to be its position in the alphabet).

once two sorted strings are achieved, in order to be anagrams, these two sorted strings should be exactly the same, which can be easily checked by iterating through both strings and checking whether all corresponding characters are equal to each other or not.

Exercise 7. *Describe an algorithm that determines whether a function from a finite set to another finite set is injective.*

The only thing that we need to show, is that given a function f and a finite list of input values $a_1, a_2, ..., a_n$ (the domain), all the respective output values of the function (i.e. $f(a_1), f(a_2), ..., f(a_n)$) are unique or not. In other word, for any distinct i and j where $1 \leq i \leq n$ and $1 \leq j \leq n$, we should have $f(a_i) \neq f(a_j)$ to have an injective function.

The following algorithm tries to check if all input pairs have distinct outputs to determine if the function is injective or not.

```
Truth = True
for i = 1 to n
    for j = 1 to n
        if (a_i ≠ a_j and f(a_i) = f(a_j))
```

```
            Truth = False
    return Truth
```

Exercise 8. *We use Bubble Sort, Selection Sort, and Insertion Sort to sort the following sequence (see Exercise 4):*

`9, 12,-43, 20, -2, 3, 7, 28, 19.`

1. *How many comparisons are done in each of the algorithms?*

   (a) **Bubble sort**:
   The basic version of the algorithm performs $\frac{n(n-1)}{2} = 36$ comparisons. In the optimized version of the algorithm, the number of comparisons can be reduced to 26.

   (b) **Selection sort**:
   The algorithm always performs $n-1$ comparisons at the first round and does one less per round, hence $\sum_{i=1}^{9-1} i = 36$ comparisons in the nested for loop. For each i we perform one additional comparison when comparing a_i to a_{min}. Hence the final number of comparisons is $36 + 8 = 44$.

   (c) **Insertion sort**:
   The algorithm needs 31 comparisons for the given list of values. To get this value, you can look at the solution of exercise 1.3 and sum up the values of the lowest i. This corresponds to the number of comparisons that were made for each value of j.

2. *How many swaps are done in each of the algorithms?*

   (a) **Bubble sort**:
   The algorithm performs a swap each time an element is bigger than its neighbor, hence, in our case, $5 + 5 + 3 = 13$ swaps.

   (b) **Selection sort**:
   The algorithm performs a swap each time a lower element is found per round, hence, in our case, 7 swaps.

   (c) **Insertion sort**:
   The algorithm does not swap but inserts the lowest element at the end of the sorted sublist, which requires, in our case, $3 + 4 + 4 + 4 + 3 = 18$ insertions.

3. *In the worst-case scenario, what is the order of how many comparisons and how many swaps are done by the three algorithms for an input sequence of length $n + 1$?*

   Finding a worst-case scenario is not a trivial task, however finding the worst-case order of operations is usually easier. For example, loops will be executed until the end instead of being stopped early due to conditional statements. For the algorithms in the exercise:

   (a) **Bubble sort**:
   The loops do not have early termination conditions, so they will always execute completely. The outer loop produces $(n+1)-1$ executions of the inner loop, which in turn will execute its contents $(n+1)-1$ times the first time, then $(n+1)-2$ times, then $(n+1)-3$, and so on. As a consequence, the comparison operation in the `if` statement is always executed $\frac{n(n+1)}{2}$ times, which is a quadratic function of n. Any quadratic function of n is a correct answer to the question, since we only ask about the order of operations and not the precise number. The swap operation, instead, is only executed if the `if` condition is true. Thus, to analyze the worst case, we assume that it's always true, originating $\frac{n(n+1)}{2}$ swaps. Since we only ask about order, any quadratic function is a correct answer. Notice that the comparisons are always the same number, whereas the swaps will be this many only in the worst case.

(b) **Selection sort**:

Similarly to the previous case, the outer loop will always execute its contents $(n+1)-1$ times. The inner loop will also execute $(n+1)-1$ times the first time, then $(n+1)-2$ times, then $(n+1)-3$, and so on. The inner loop contains an if statement with a comparison operation, so this comparison will be performed, just as in Bubble sort, $\frac{n(n+1)}{2}$ times. Selection sort also has another comparison operation in the outer loop, which will be executed $(n+1)-1$ times. As you will see in the next weeks, this operation does not impact the overall order of how many operations are performed, so we can disregard it. Intuitively, the order of this operation grows linearly with n, whereas the other operation grows quadratically. As a consequence, the order of how many comparisons are performed by Selection sort is quadratic. For the swap operation we notice that, differently from Bubble sort, it is executed in the outer loop. This means that, in the worst case, it will be executed $(n+1)-1$ times, so again quadratically. Notice that the comparisons are always the same number, whereas the swaps are this many only in the worst case.

(c) **Insertion sort**:

Similarly to the previous case, the outer loop will always execute its contents $(n+1)-1$ times. Now, however, we have two inner loops. The first one has a comparison operation, whereas the second one does not, so we consider only the first loop. In the worst case, the comparison operation will always return `true` until `i == j`, in which case a_j cannot be greater than itself. This means that the comparison is executed 2 times during the first execution of the outer loop, then 3 times, then 4, and so on until it is executed $(n+1)$ times. The total number of comparisons is thus $\frac{(n+1)(n+2)}{2} - 1$. While this number looks slightly bigger than the other two algorithms, it is still quadratic, so the order of how many comparisons is the same. Any quadratic function of n is a correct answer. Regarding swaps, the provided pseudo-code of Insertion sort does not use swap operations, but uses element insertions. Counting insertions, we notice that they are executed inside the second inner loop and they are also executed in the outer loop. The second inner loop is executed $j-i-1$ times. The worst case happens when i is the smallest possible, which is 1. This makes the second inner loop execute $j-1$ times, so it will execute 1 time, then 2 times, then 3, and so on until it is executed $(n+1)-1$ times. This makes the total number of insertions $\frac{n(n+1)}{2}$. The insertions that happen in the outer loop will originate a number of insertions that grows linearly with n, so it can be disregarded in this analysis. The worst-case number of insertions grows quadratically with n.

Algorithm	Comp.	Swaps
Bubble sort	$\frac{n(n+1)}{2}$	$\frac{n(n+1)}{2}$
Selection sort	$\frac{n(n+1)}{2}$	n
Insertion sort	$\frac{n(n+1)}{2}$	$\frac{n(n+1)}{2}$